PROJECT PROPOSAL: AUTOMATED API THREAT DETECTION AND SECURITY ANOMALY LOGGING ENGINE

Course: INSY 8311 – Database Development with PL/SQL

Instructor: Eric Maniraguha

Student Name: Ishimwe Kevin

Student ID: 20251SEN190

GitHub Repository: https://github.com/ikevin01/kevin_20251SEN190_apiThreatLog

1. Problem Statement

Modern microservice architectures and web applications rely heavily on Application Programming Interfaces (APIs) to handle sensitive financial, institutional, and administrative data transactions. However, these systems face severe security vulnerabilities from automated bot attacks, rate-limit abuse, credential stuffing, and injection payloads.

Standard security mechanisms typically log suspicious activity at the application or web server layer. This creates a critical delay between threat detection and database-level data isolation. When application servers fail or are bypassed, database backends remain vulnerable to unauthorized access and log tampering. There is a clear operational need for an autonomous database security engine within Oracle PL/SQL that can independently ingest API request traffic, compute threat severity scores, and isolate malicious actors in real time directly at the database layer.

2. Proposed Solution

This project develops an **Automated API Threat Detection and Security Anomaly Logging Engine** inside the Oracle Pluggable Database ishimwe_kevindb. The core solution provides an active security layer that evaluates incoming API request metrics, monitors rate-limit thresholds, and tracks HTTP error frequencies.

Using PL/SQL packages, functions, and database triggers, the system automatically detects abnormal traffic patterns. When an incoming request exceeds allowed rate thresholds or contains unauthorized signature traits, the database fires an autonomous security trigger. This trigger logs the event into an audit repository and immediately inserts the source IP address into an active quarantine list, blocking further operations without requiring application-tier intervention.

3. Data Source

The primary dataset used for this project is the **Cybersecurity Threat Detection Logs** hosted on Kaggle
([https://www.kaggle.com/datasets/aryan208/cybersecurity-threat-detection-logs](https://www.ka

ggle.com/datasets/aryan208/cybersecurity-threat-detection-logs)). This dataset contains over 100,000 records of structured network and web API traffic logs. Key data attributes extracted include source_ip, request_path, http_method, response_code, payload_bytes, timestamp, and threat_label. The dataset will be preprocessed and imported into Oracle 19c/21c staging tables using SQL*Loader and PL/SQL data loading routines.

4. Planned Tables (3NF Schema)

The database schema is designed in Third Normal Form (3NF) to ensure data integrity and query efficiency:

1. API_CLIENTS (client_id [PK], client_name, api_key_hash, account_status, created_at) – Stores registered API consumers and authentication status.
2. API_ENDPOINTS (endpoint_id [PK], endpoint_path, http_method, rate_limit_per_min, risk_classification) – Maps API routes and their strict throughput rules.
3. REQUEST_LOGS (log_id [PK], client_id [FK], endpoint_id [FK], ip_address, response_code, payload_bytes, request_timestamp) – Ingests raw API traffic events.
4. SECURITY_THREATS (threat_id [PK], log_id [FK], ip_address, threat_type, severity_score, detected_at) – Records identified attack signatures.
5. IP_QUARANTINE_LIST (quarantine_id [PK], ip_address, quarantine_reason, blocked_until, is_active) – Maintains active IP blocklists managed by automated triggers.

5. Planned PL/SQL Features

- **Packages:** PKG_API_SECURITY – Encapsulates log ingestion procedures, IP quarantine logic, and security status lookups.
- **Stored Procedures:** PRC_LOG_API_REQUEST (validates client tokens and logs inbound payload metrics) and PRC_QUARANTINE_IP (manages automated IP locking).
- **Stored Functions:** FNC_CALCULATE_RISK_SCORE – Evaluates request volume and HTTP 4xx/5xx error frequencies within rolling time windows to return a dynamic risk score (0–100).
- **Database Triggers:** TRG_DETECT_ANOMALY – An AFTER INSERT trigger on REQUEST_LOGS that fires upon entry to evaluate risk scores and execute autonomous quarantines (PRAGMA AUTONOMOUS_TRANSACTION).
- **Cursors:** Explicit cursor blocks used in batch monitoring and daily security audit reporting.

6. Planned Innovation

- **Category:** Security Monitoring (Option 3).
- **Implementation Tool:** Grafana connected live to Oracle Database via Oracle REST Data Services (ORDS) / SQL driver.
- **Functionality:** An interactive, live operational dashboard visualizing real-time database security metrics. The dashboard will display key metrics including Total API Requests

per Minute, Live Threat Incident Spikes, Blocked Malicious IP Counters, and real-time security audit logs driven directly by PL/SQL database execution.

7. References (APA)

- Al-Mhiqani, M. N., et al. (2020). Cyber-threat detection and mitigation in API gateways: A review. *IEEE Access*, 8, 120412-120426.
- Amazon Web Services. (2025). *AWS WAF Developer Guide: Rate-based rules and IP reputation list enforcement*. AWS Documentation.
- Gupta, S., & Kumar, R. (2022). Automated IP quarantine and threat mitigation architectures in enterprise relational databases. *Journal of Cybersecurity Technology*, 6(3), 145-162.
- Kaggle Security Datasets. (2024). *Cybersecurity Threat Detection Logs*. Kaggle Dataset Repository.
- Oracle Corporation. (2024). *Database PL/SQL Language Reference (19c / 21c)*. Oracle Help Center.
- OWASP Foundation. (2023). *OWASP API Security Top 10 2023*. OWASP.